

If different classes have the same behavior, you may want to combine the classes through inheritance (Extract Superclass and Extract Subclass)

- Payoff
 - Improves code organization
 - Reduces code duplication
 - Simplifies support

Divergent Change (تغییر واگرا)

تعریف ساده

Divergent Change شبیه Shotgun Surgery هست ولی در واقع برعکسشه:

- Divergent Change = تغییرات زیاد روی یه کلاس واحد
- Shotgun Surgery = یه تغییر واحد روی چندین کلاس

علائم و نشانه‌ها (Signs and Symptoms)

مجبوری متدهای نامرتبب زیادی رو توی یه کلاس تغییر بدی. مثلاً وقتی یه نوع محصول جدید اضافه می‌کنی، باید متدهای جستجو، نمایش، و سفارش محصول رو تغییر بدی.

دلایل ایجاد (Reasons for the Problem)

معمولاً به خاطر ساختار ضعیف برنامه یا برنامه نویسی Copy-Paste ایجاد میشه.

مثال 

❌ // کلاسی که با هر تغییر، متدهای نامرتبطش باید عوض بشن

```
public class ProductManager
{
    // اگر نوع محصول جدید اضافه بشه، اینا باید تغییر کنن:
    { /* ... */ } public List<Product> FindProducts(string type)
    { /* ... */ } public void DisplayProduct(Product p)
    { /* ... */ } public void OrderProduct(Product p, int qty)

    // اگر روش ارسال جدید اضافه بشه، اینا باید تغییر کنن:
    { /* ... */ } public decimal CalculateShipping(string method)
    { /* ... */ } public void PrintShippingLabel(string method)

    // اگر روش پرداخت جدید اضافه بشه، اینا باید تغییر کنن:
    { /* ... */ } public void ProcessPayment(string paymentType)
    { /* ... */ } public void RefundPayment(string paymentType)
}
```

درمان (Treatment)

#	تکنیک	کاربرد
۱	Extract Class	رفتارهای نامرتبط رو به کلاس‌های جدا منتقل کن
۲	Extract Superclass / Extract Subclass	اگر کلاس‌های مختلف رفتار مشترک دارن، با ارث‌بری ترکیبشون کن

Payoff (دستاورد)

- سازماندهی بهتر کد
- کاهش کد تکراری
- پشتیبانی ساده‌تر

تمرین

این ProductManager رو با درمان مناسب ریفکتور کن:

csharp

```

public class ProductManager
{
    // بخش جستجو و نمایش (برای محصول)
    public List<Product> FindProducts(string type)
    {
        // شبیه‌سازی
        return new List<Product>();
    }

    public void DisplayProduct(Product p)
    {
        Console.WriteLine($"Product: {p.Name}")
    }

    // بخش ارسال (کاملاً نامرتبط)
    public decimal CalculateShipping(string method)
    {
        return method == "Express" ? 50000 : 15000
    }

    public void PrintShippingLabel(string method)
    {
        Console.WriteLine($"Shipping: {method}")
    }

    // بخش پرداخت (کاملاً نامرتبط)
    public void ProcessPayment(string paymentType)
    {
        Console.WriteLine($"Payment: {paymentType}")
    }

    public void RefundPayment(string paymentType)
    {
        Console.WriteLine($"Refund: {paymentType}")
    }
}

```

راهنمایی: با **Extract Class** به سه کلاس بشکن:

DisplayProduct , FindProducts ← ProductRepository •
PrintShippingLabel , CalculateShipping ← ShippingService •
RefundPayment , ProcessPayment ← PaymentService •

بفرست تا بررسی کنیم! 🤖